# Contents


**1 version 1** . . . . . **1**

**2 version 2** . . . . . **3**

2.0.1 平面几何 . . . . . 3

2.0.2 自适应辛普森法 . . . . . 19

2.0.3 平面最近点对 . . . . . 19

2.0.4 最小圆覆盖和最小球覆盖 . . . . . 20


## 1 version 1

```
typedef long double T;
const T EPS = 1e-12;

struct pt // 点/向量
{
    T x,y;
    pt(const T& x_ = 0,const T& y_ = 0):x(x_),y(y_){}
    pt operator+(pt p){return {x+p.x,y+p.y};}
    pt operator-(pt p){return {x-p.x,y-p.y};}
    pt operator*(T d){return {x*d,y*d};}
    pt operator/(T d){return {x/d,y/d};}
    bool operator==(const pt& b){
        return std::abs(x - b.x) < EPS && std::abs(y - b.y) < EPS;
    }
    bool operator!=(const pt& b){return !(*this == b);}
    bool operator<(const pt& b) const {
        if (std::abs(x - b.x) > EPS) return x < b.x;
        return y < b.y - EPS;
    }
    void read() {
        std::cin>>x>>y;
    }
    void print() {
        std::cout<<std::fixed<<std::setprecision(10)<<x<<" "<<y<<"\n";
    }
};

int sign(T x) // return x == 0
{
    return (x > EPS) - (x < -EPS);
}
T sq(pt p)// 向量模的平方
{return p.x*p.x+p.y*p.y;}
T absv(pt p) // 返回向量的模
{return sqrtl(sq(p));}
pt scale(pt c,T factor,pt p)// 以 c 为中心将 p 缩放 factor 倍
{
    return c+(p-c)*factor;
}
pt rot(const pt& p,T a) // 将向量p逆时针旋转角度a(弧度制)
{
    return {p.x*cosl(a)-p.y*sinl(a),p.x*sinl(a)+p.y*cosl(a)};
}
pt perp(const pt& p) // 返回p的垂线向量(逆时针旋转90度)
{
    return {-p.y,p.x};
}
// 以下(大写字母表示向量)
// 点积: V*W = |v|*|w|*cos(angle)
T dot(pt v,pt w){ return v.x*w.x+v.y*w.y;}

bool isPerp(const pt& v,const pt& w) // 判断两个向量是否垂直
// {return std::abs(dot(v,w)) < 1e-9;}
{return !sign(dot(v,w));}


// 叉积: A*B = |a|*|b|*sin(angle)
// v到w的夹角,cross(v, w) > 0 ==> sin(angle) > 0 ==> w 在 v 的逆时针方向
T cross(pt v,pt w){return v.x*w.y-v.y*w.x;}

T angle(pt v,pt w) // 计算两个向量的夹角
{ // tan = sin/cos
    return std::abs(atan2l(cross(v,w), dot(v,w)));
}
T orient(pt a,pt b,pt c) // 判断点c相对与线段ab的位置
// 正值表示在左侧(逆时针), 负值表示在右侧顺时针, 0表示三点共线
{ return cross(b-a, c-a);}
```

```
bool inAngle(pt a,pt b,pt c,pt p) // 判断点p是否在角BAC范围内
{
    // assert(orient(a, b, c) != 0);
    if(orient(a, b, c)<0) std::swap(b,c);
    return orient(a, b, p)>=0 && orient(a, c, p)<=0;
}
bool half(pt p)// 极角排序，用于区分上半平面和下半平面
{
    // return p.y>0 || (std::abs(p.y) < 1e-9 && p.x>0);
    return p.y>0 || (!sign(p.y) && p.x>0);
}
bool cmpPolar(pt x,pt y) // 极角排序
{
    int tx = half(x),ty = half(y);
    if(tx!=ty) return tx<ty;
    else return x.y*y.x<y.y*x.x;
}

struct line{
    pt v;T c; // v 为直线的方向向量
    line(pt v_,T c_):v(v_),c(c_){}
    // 直线方程a*x + b*y + c = 0,方向向量(b,-a)
    line(T a,T b,T c):v({b,-a}),c(-c){}
    line(pt p,pt q):v(q-p),c(cross(v, p)){}
};
T side(line ln,pt p)// 判断p在直线ln的哪一侧
{return cross(ln.v,p)-ln.c;} // a*x+b*y-c    正负是方向向量v转到p的旋转方向
T dist(line ln,pt p) // p到直线ln的垂直距离
{return std::abs(side(ln,p))/absv(ln.v);}
T sqDist(line ln,pt p) // 点到直线距离的平方
{return side(ln,p)*side(ln,p)/sq(ln.v);}
line perpThrough(line ln,pt p)// 返回过点p垂直于ln的直线
{ return {p,p+perp(ln.v)};}
bool inter(line l1,line l2,pt &out)// 求两直线交点，返回false表示两直线平行
{
    T d = cross(l1.v, l2.v);
    // if(std::abs(d) < 1e-9) return false;
    if(!sign(d)) return false;
    out = (l2.v*l1.c-l1.v*l2.c)/d;
    return true;
}

bool cmpProj(line ln,pt p,pt q){ // 根据点在直线上的投影排序
    return dot(ln.v,p) < dot(ln.v,q);
}
line translate(line ln,pt t){ // 把直线平移向量 t
    // a(x-t.x)*b(y-t.y)=c   a*t.x + b*t.y = cross(ln.v,t)
    return {ln.v,ln.c+cross(ln.v,t)};
}
pt proj(line ln,pt p){ // 点p在直线ln上的正交投影
    return p-perp(ln.v)*side(ln,p)/sq(ln.v); // perp(ln.v)*side(ln,p)/sq(ln.v)是垂点到p的向量z
}
pt refl(line ln,pt p){ // p关于直线ln的反射投影(对称点)
    return p-perp(ln.v)*2*side(ln, p)/sq(ln.v);
}
bool inDisk(pt a,pt b,pt p){ // 判断点p是否在直径为ab的圆内
    return dot(a-p, b-p)<=0; // 原理:夹角<=90,sin(angle)<=0
}
bool onSegment(pt a,pt b,pt p){ // 判断点p是否在线段ab
    // return std::abs(orient(a, b, p)) < 1e-9 && inDisk(a, b, p);
    return !sign(orient(a, b, p)) && inDisk(a, b, p);
}
bool properInter(pt a,pt b,pt c,pt d,pt& out)
// 判断线段ab和cd是否在非端点处相交(严格相交判定)
{
    T oa = orient(c, d, a),ob = orient(c, d, b),
        oc = orient(a, b, c),od = orient(a, b, d);
    if(oa*ob<0 && oc*od<0){
        out = (a*ob-b*oa)/(ob-oa);
        return true;
    }
    return false;
}
std::set<pt> inters(pt a,pt b,pt c,pt d)
// 通用线段相交，返回所有交点（0(不相交)/1(相交),如果线段重合返回重合的两个端点）
{
    pt out;
    if(properInter(a, b, c, d, out)) return {out};
    std::set<pt> res;
```

```
    if(onSegment(c, d, a)) res.insert(a);
    if(onSegment(c, d, b)) res.insert(b);
    if(onSegment(a, b, c)) res.insert(c);
    if(onSegment(a, b, d)) res.insert(d);
    return res;
// res.size() == 1表示两个线段的交点，res.size() == 2表示相交区域是线段，其中存的是这个线段的两个端点
}
T segPoint(pt a,pt b,pt p)//点p到线段ab的距离，如果垂足不在内部，取到端点的距离
{
    if(a != b){
        line l(a,b);
        if(cmpProj(l,a,p) && cmpProj(l, p, b))
            return dist(l, p);
        return std::min(absv(p-a),absv(p-b));
    }
    return absv(p-a);
}
T segSeg(pt a,pt b,pt c,pt d)// 线段到线段的距离
// 相交为0,否则到为四个端点到对方线段距离的最小值
{
    pt dummy;
    if(properInter(a, b, c, d, dummy)) return 0;
    return std::min({segPoint(a, b, c),segPoint(a, b, d),
        segPoint(c, d, a),segPoint(c, d, b)});
}
T areaPolygon(std::vector<pt> p)
// 多边形的点需按顺序排列,返回多边形的面积
{
    T area = 0.0;
    for(int i = 0,n = p.size();i<n;i++){
        area += cross(p[i], p[(i+1)%n]);
    }
    return std::abs(area)/2.0;
}
bool above(pt a,pt p)// 判断点p是否在点a的上方
{
    return p.y >= a.y;
}
bool crossesRay(pt a,pt p,pt q)
// 判断a向右发出的射线(不包含点)是否穿过线段pq
{
    return (above(a, q)-above(a, p))*orient(a, p, q) > 0;
}
bool inPolygon(std::vector<pt> p,pt a,bool strict = true)
// 判断点是否在多边形内部,strict 是判断是否严格在多边形内部
{
    int numCrossings = 0;
    for(int i = 0,n = p.size();i<n;i++){
        if(onSegment(p[i], p[(i+1)%n], a)) return !strict;
        numCrossings += crossesRay(a, p[i], p[(i+1)%n]);
    }
    return numCrossings&1;
}
```

## 2 version 2

我们设两个点 $(x1, y1), (x2, y2)$

两点的曼哈顿距离:$|x_1 - x_2| + |y_1 - y_2|$

两点的切比雪夫距离:$max(|x_1 - x_2| + |y_1 - y_2|)$

两点的欧几里得距离:$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

曼哈顿与切比雪夫距离的转化:

曼哈顿 ==> 切比雪夫:$(x, y)$ ==> $(x + y, x - y)$

切比雪夫 ==> 曼哈顿: $(x, y)$ ==> $(\frac{x+y}{2}, \frac{x-y}{2})$

### 2.0.1 平面几何

$std :: sin, std :: cos, std :: tan$ 返回对应三角函数值，有 $float, double, long\ double$ 三种返回值，和传参的类型相同。注意一定是 $std ::$ 前缀下的函数

注意 $std :: acos, std :: asin$ 的传参范围是 $[-1, 1]$ 注意不要越界，$std :: tan$ 不能传入 $\frac{\pi}{2} + k\pi$

注意 $std :: atan2(y, x)$ 的返回值在 $(-\pi, \pi]$, 返回的是 从正 x 轴到点 (x,y) 的夹角（弧度）

具体的说，若点 $(x, y)$ 在第一二象限则返回值为正，返回值按逆时针方向绝对值逐渐增大

否则返回值为负，返回值按顺时针方向绝对值逐渐增大。

```cpp
#include <bits/stdc++.h>

using ll = long long;
using ld = long double;  // 浮点数类型
const ld eps = 1e-10;    // 浮点精度阈值
const ld pi = acosl(-1); // PI
using pld = std::pair<ld, ld>;
using pii = std::pair<int, int>;
using pll = std::pair<ll, ll>;

// 符号函数：返回x的符号（1/0/-1）
int sgn(ld x)
{
    if (x > eps)
        return 1;
    if (x < -eps)
        return -1;
    return 0;
}
// 基础二维点/向量结构
template <class T>
struct Point
{
    T x, y;
    // 向量运算
    bool operator==(const Point &a) const { return (fabs(x - a.x) <= eps && fabs(y - a.y) <= eps); }
    bool operator<(const Point &a) const
    {
        if (fabs(x - a.x) <= eps)
            return y < a.y - eps;
        return x < a.x - eps;
    }
    bool operator>(const Point &a) const { return !(*this < a || *this == a); }
    Point operator+(const Point &a) const { return {x + a.x, y + a.y}; } // 向量加
    Point operator-(const Point &a) const { return {x - a.x, y - a.y}; } // 向量减
    Point operator-() const { return {-x, -y}; }                         // 向量取反
    Point operator*(const T k) const { return {k * x, k * y}; }
    Point operator/(const T k) const { return {x / k, y / k}; }
    // 点积（*）、叉积（%）
    T operator*(const Point &a) const { return x * a.x + y * a.y; }                  // 点积
    T operator%(const Point &a) const { return x * a.y - y * a.x; }                  // 叉积
    T len2() const { return (*this) * (*this); }                                     // 向量长度的平方
    T dis2(const Point &a) const { return (a - (*this)).len2(); }                    // 两点距离的平方
    ld len() const { return sqrtl(len2()); }                                         // 向量长度
    ld dis(const Point &a) const { return sqrtl(dis2(a)); }                          // 两点距离
    ld ang(const Point &a) const
{ return acosl(std::max(-1.0l, std::min(1.0l, ((*this) * a) / (len() * a.len())))); } // 向量夹角
    int toleft(const Point &a) const
    {
        const auto t = (*this) % a;
        return (t > eps) - (t < -eps);
    } // to-left 测试
    // 检查a向量在this的哪一测 >0逆时针 <0顺时针 =0同向

    // 旋转：90度逆时针
    Point rotate90() const { return {-y, x}; } // 逆时针旋转90°
    Point rotate(ld theta) const
    {
        ld c = cosl(theta), s = sinl(theta);
        return {
            (x * c - y * s),
            (x * s + y * c)};
    }
    Point rotate(const ld cosr, const ld sinr) const
    { return {x * cosr - y * sinr, x * sinr + y * cosr}; }
    // 逆时针旋转（给定角度的正弦与余弦）
};
// 极角排序
template <class T>
struct argcmp
{
    bool operator()(const Point<T> &a, const Point<T> &b) const
    {
        const auto quad = [](const Point<T> &a)
        {
            if (a.y < -eps)
                return 1;
            if (a.y > eps)
```

```
                return 4;
            if (a.x < -eps)
                return 5;
            if (a.x > eps)
                return 3;
            return 2;
        };
        const int qa = quad(a), qb = quad(b);
        if (qa != qb)
            return qa < qb;
        const auto t = a % b;
        if (fabs(t) <= eps)
            return a * a < b * b - eps; // 不同长度的向量需要分开
        return t > eps;
    }
};
template <class T>
struct Line // 直线
{
    Point<T> p, v; // p 为直线上一点，v 为方向向量

    bool operator==(const Line &a) const { return v.toleft(a.v) == 0 && v.toleft(p - a.p) == 0; }
    int toleft(const Point<T> &a) const { return v.toleft(a - p); } // to-left 测试 点a和该直线的位置关系
    bool operator<(const Line &a) const                             // 半平面交算法定义的排序
    {
        if (fabs(v % a.v) <= eps && v * a.v >= -eps)
            return toleft(a.p) == -1;
        return argcmp<T>()(v, a.v);
    }
    // 涉及浮点数
    ld dis(const Point<T> &a) const { return fabs(v % (a - p)) / v.len(); } // 点到直线距离
    // --------------------------------
    // 求两直线交点
    // 返回值：是否有唯一交点
    //   false: 平行或重合
    //   true : 有交点
    // --------------------------------
    bool isCross(const Line &a) const // 是否有交
    {
        ld d = v % a.v;
        return sgn(d) ? 1 : 0;
    }
    Point<T> inter(const Line &a) const // 求交
    {
        ld d = v % a.v;
        ld t = (a.v % (p - a.p)) / d;
        return p + v * t;
    }
    // --------------------------------
    // 点到直线的垂足
    // 参数：q 点
    // 返回：q 在直线上的投影点（垂足）
    // --------------------------------
    Point<T> foot(const Point<T> &q) const // 垂足
    {
        // 向量 p->q
        Point<T> u = q - p;
        // 投影系数：t = (u • v) / (v • v)
        ld t = 1.L * (u * v) / (v * v);
        return p + v * t;
    }
    // 点a关于直线bc的对称点
    Point<T> symmetry(Point<T> a) const
    {
        // 先求垂足
        Point<T> f = foot(a);
        // 对称点：f + (f - a) = 2f - a
        return f * 2 - a;
    }
};
template <class T>
struct Segment
{
    Point<T> a, b;
    // 判断点a是否在线段bc上
    //-1在端点上 0不在 1在线段中间
    int onSeg(Point<T> p) const
    {
        if (p == a || p == b)
```

```
            return -1;
        return (p - a).toleft(p - b) == 0 && (p - a) * (p - b) < -eps;
    }
    // 判断线段直线是否相交
    // -1 直线经过线段端点 / 0 线段和直线不相交 / 1 线段和直线严格相交
    int isCross1(const Line<T> &c) const
    {
        if (c.toleft(a) == 0 || c.toleft(b) == 0)
        {
            return -1;
        }
        return c.toleft(a) != c.toleft(b);
    }
    Point<T> inter1(const Line<T> &c) const // 需要保证有交
    {
        if (c.toleft(a) == 0)
        {
            return a;
        }
        if (c.toleft(b) == 0)
        {
            return b;
        }
        ld cur = 1.L * (c.v % (c.p - a)) / (c.v % (b - a));
        return a + cur * (b - a);
    }
    // 判断两线段是否相交
    // -1 在某一线段端点处相交 / 0 两线段不相交 / 1 两线段严格相交
    int isCross2(const Segment &s) const
    {
        if (onSeg(s.a) || onSeg(s.b) || s.onSeg(a) || s.onSeg(b))
        {
            return -1;
        }
        Line<T> l{a, b - a}, ls{s.a, s.b - s.a};
        return l.toleft(s.a) * l.toleft(s.b) == -1 && ls.toleft(a) * ls.toleft(b) == -1;
    }
    Point<T> inter2(const Segment &s) const // 需要保证有交
    {
        if (onSeg(s.a))
        {
            return s.a;
        }
        if (onSeg(s.b))
        {
            return s.b;
        }
        if (s.onSeg(a))
        {
            return a;
        }
        if (s.onSeg(b))
        {
            return b;
        }
        Point<T> u = b - a;     // 向量 AB
        Point<T> v = s.b - s.a; // 向量 CD
        Point<T> w = s.a - a;   // 向量 AC
        // t = (AC × v) / (u × v)
        ld t = 1.L * (w % v) / (u % v);
        return a + t * u;
    }
    // 点到线段距离
    ld dis(const Point<T> &p) const
    {
        if ((p - a) * (b - a) < -eps || (p - b) * (a - b) < -eps)
        {
            return std::min(p.dis(a), p.dis(b));
        }
        const Line<T> l{a, b - a};
        return l.dis(p);
    }
    // 两线段间最近两点的距离
    ld dis(const Segment &s) const
    {
        if (isCross2(s))
        {
            return 0;
        }
        return std::min({dis(s.a), dis(s.b), s.dis(a), s.dis(b)});
```

```
    }
};
template <class T>
struct Polygon
{
    std::vector<Point<T>> p;
    int nxt(int t) const
    {
        return t == p.size() - 1 ? 0 : t + 1;
    }
    int pre(int t) const
    {
        return t == 0 ? p.size() - 1 : t - 1;
    }
    std::pair<bool, int> winding(const Point<T> &a) const
    {
        int cnt = 0;
        for (int i = 0; i < p.size(); i++)
        {
            const Point<T> u = p[i], v = p[nxt(i)];
            if (fabs((a - u) % (a - v)) <= eps && (a - u) * (a - v) <= eps)
                return {true, 0};
            if (fabs(u.y - v.y) <= eps)
                continue;
            const Line<T> uv = {u, v - u};
            if (u.y < v.y - eps && uv.toleft(a) <= 0)
                continue;
            if (u.y > v.y + eps && uv.toleft(a) >= 0)
                continue;
            if (u.y < a.y - eps && v.y >= a.y - eps)
                cnt++;
            if (u.y >= a.y - eps && v.y < a.y - eps)
                cnt--;
        }
        return {false, cnt};
    }

    // 0表示在上面  -1表示在外面 1表示在里面
    int operator()(const Point<T> &a) const // 点是否在多边形内 回转数法
    {
        auto [onEdge, cnt] = winding(a);
        if (onEdge)
            return 0;
        return (cnt != 0 ? 1 : -1);
    }
    // 多边形面积的两倍
    // 可用于判断点的存储顺序是顺时针或逆时针
    T area() const
    {
        T sum = 0;
        for (int i = 0; i < p.size(); i++)
            sum += p[i] % p[nxt(i)];
        return sum;
    }
    // 多边形的周长
    ld circ() const
    {
        ld sum = 0;
        for (int i = 0; i < p.size(); i++)
            sum += p[i].dis(p[nxt(i)]);
        return sum;
    }
};

template <class T>
struct Convex
{
    std::vector<Point<T>> p;
    int size() const
    {
        return p.size();
    }
    int nxt(int t) const
    {
        return t == p.size() - 1 ? 0 : t + 1;
    }
    int pre(int t) const
    {
        return t == 0 ? p.size() - 1 : t - 1;
    }
```

```
    Convex operator+(const Convex &c) const
    {
        const auto &p = this->p;
        std::vector<Segment<T>> e1(p.size()), e2(c.p.size()), edge(p.size() + c.p.size());
        std::vector<Point<T>> res;
        res.reserve(p.size() + c.p.size());
        const auto cmp = [](const Segment<T> &u, const Segment<T> &v)
        { return argcmp<T>()(u.b - u.a, v.b - v.a); };
        for (int i = 0; i < p.size(); i++)
            e1[i] = {p[i], p[this->nxt(i)]};
        for (int i = 0; i < c.p.size(); i++)
            e2[i] = {c.p[i], c.p[c.nxt(i)]};
        std::rotate(e1.begin(), std::min_element(e1.begin(), e1.end(), cmp), e1.end());
        std::rotate(e2.begin(), std::min_element(e2.begin(), e2.end(), cmp), e2.end());
        std::merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
        const auto check = [](const std::vector<Point<T>> &res, const Point<T> &u)
        {
            const auto b1 = res.back(), b2 = *std::prev(res.end(), 2);
            return (b1 - b2).toleft(u - b1) == 0 && (b1 - b2) * (u - b1) >= -eps;
        };
        auto u = e1[0].a + e2[0].a;
        for (const auto &v : edge)
        {
            while (res.size() > 1 && check(res, u))
                res.pop_back();
            res.push_back(u);
            u = u + v.b - v.a;
        }
        if (res.size() > 1 && check(res, res[0]))
            res.pop_back();
        return {res};
    }
    // 旋转卡壳
    // func 为更新答案的函数，可以根据题目调整位置
    template <typename F>
    void rotcaliper(const F &func) const
    {
        const auto &p = this->p;
        const auto area = [](const Point<T> &u, const Point<T> &v, const Point<T> &w)
        { return (w - u) % (w - v); };
        for (int i = 0, j = 1; i < p.size(); i++)
        {
            const auto nxti = this->nxt(i);
            func(p[i], p[nxti], p[j]);
            while (area(p[this->nxt(j)], p[i], p[nxti]) >= area(p[j], p[i], p[nxti]))
            {
                j = this->nxt(j);
                func(p[i], p[nxti], p[j]);
            }
        }
    }

    // 凸多边形的直径的平方
    T diameter2() const
    {
        const auto &p = this->p;
        if (p.size() == 1)
            return 0;
        if (p.size() == 2)
            return p[0].dis2(p[1]);
        T ans = 0;
        auto func = [&](const Point<T> &u, const Point<T> &v, const Point<T> &w)
        {
            ans = std::max({ans, w.dis2(u), w.dis2(v)});
        };
        rotcaliper(func);
        return ans;
    }
    // 判断点是否在凸多边形内
    // 复杂度 O(logn)
    // -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
    int operator()(const Point<T> &a) const
    {
        const auto &p = this->p;
        if (p.size() == 1)
            return a == p[0] ? -1 : 0;
        if (p.size() == 2)
            return Segment<T>{p[0], p[1]}.onSeg(a) ? -1 : 0;
        if (a == p[0])
            return -1;
```

```
        if ((p[1] - p[0]).toleft(a - p[0]) == -1 || (p.back() - p[0]).toleft(a - p[0]) == 1)
            return 0;
        const auto cmp = [&](const Point<T> &u, const Point<T> &v)
        { return (u - p[0]).toleft(v - p[0]) == 1; };
        const int i = std::lower_bound(p.begin() + 1, p.end(), a, cmp) - p.begin();
        if (i == 1)
            return Segment<T>{p[0], p[i]}.onSeg(a) ? -1 : 0;
        if (i == p.size() - 1 && Segment<T>{p[0], p[i]}.onSeg(a))
            return -1;
        if (Segment<T>{p[i - 1], p[i]}.onSeg(a))
            return -1;
        return (p[i] - p[i - 1]).toleft(a - p[i - 1]) > 0;
    }

    // 凸多边形关于某一方向的极点
    // 复杂度 O(logn)
    // 参考资料：https://codeforces.com/blog/entry/48868
    template <typename F>
    int extreme(const F &dir) const
    {
        const auto &p = this->p;
        const auto check = [&](const int i)
        { return dir(p[i]).toleft(p[this->nxt(i)] - p[i]) >= 0; };
        const auto dir0 = dir(p[0]);
        const auto check0 = check(0);
        if (!check0 && check(p.size() - 1))
            return 0;
        const auto cmp = [&](const Point<T> &v)
        {
            const int vi = &v - p.data();
            if (vi == 0)
                return 1;
            const auto checkv = check(vi);
            const auto t = dir0.toleft(v - p[0]);
            if (vi == 1 && checkv == check0 && t == 0)
                return 1;
            return checkv % (checkv == check0 && t <= 0);
        };
        return std::partition_point(p.begin(), p.end(), cmp) - p.begin();
    }

    // 过凸多边形外一点求凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    // 必须保证点在多边形外
    std::pair<int, int> tangent(const Point<T> &a) const
    {
        const int i = extreme([&](const Point<T> &u)
                              { return u - a; });
        const int j = extreme([&](const Point<T> &u)
                              { return a - u; });
        return {i, j};
    }

    // 求平行于给定直线的凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    std::pair<int, int> tangent(const Line<T> &a) const
    {
        const int i = extreme([&](...)
                              { return a.v; });
        const int j = extreme([&](...)
                              { return -a.v; });
        return {i, j};
    }
};
// 点集的凸包
// Andrew 算法，复杂度 O(nlogn)
template <class T>
std::vector<Point<T>> convexhull(std::vector<Point<T>> p)
{
    std::vector<Point<T>> st;
    if (p.empty())
    {
        return std::vector<Point<T>>{};
    }
    std::sort(p.begin(), p.end());
    const auto check = [](const std::vector<Point<T>> &st, const Point<T> &u)
    {
        const auto b1 = st.back(), b2 = *prev(st.end(), 2);
        return (b1 - b2).toleft(u - b1) <= 0;
    };
```

```
    for (const auto &u : p)
    {
        while (st.size() > 1 && check(st, u))
            st.pop_back();
        st.push_back(u);
    }
    int k = st.size();
    p.pop_back();
    std::reverse(p.begin(), p.end());
    for (const auto &u : p)
    {
        while (st.size() > k && check(st, u))
            st.pop_back();
        st.push_back(u);
    }
    st.pop_back();
    return st;
}
template <class T>
struct Circle
{
    Point<T> c;
    T r;
    bool operator==(const Circle &a) const { return c == a.c && fabs(r - a.r) <= eps; }
    ld circ() const { return 2 * pi * r; } // 周长
    ld area() const { return pi * r * r; } // 面积

    // 点与圆的关系
    // -1 圆上 / 0 圆外 / 1 圆内
    int is_in(const Point<T> &p) const
    {
        const ld d = p.dis(c);
        return fabs(d - r) <= eps ? -1 : d < r - eps;
    }
    // 直线与圆关系
    // 0 相离 / 1 相切 / 2 相交
    int relation(const Line<T> &l) const
    {
        const ld d = l.dis(c);
        if (d > r + eps)
            return 0;
        if (fabs(d - r) <= eps)
            return 1;
        return 2;
    }
    // 圆与圆关系
    // -1 相同 / 0 相离 / 1 外切 / 2 相交 / 3 内切 / 4 内含
    int relation(const Circle<T> &a) const
    {
        if (*this == a)
            return -1;
        const ld d = c.dis(a.c);
        if (d > r + a.r + eps)
            return 0;
        if (fabs(d - r - a.r) <= eps)
            return 1;
        if (fabs(d - fabs(r - a.r)) <= eps)
            return 3;
        if (d < fabs(r - a.r) - eps)
            return 4;
        return 2;
    }

    // 直线与圆的交点
    std::vector<Point<T>> inter(const Line<T> &l) const
    {
        const ld d = l.dis(c);
        const Point<T> p = l.foot(c);
        const int t = relation(l);
        if (t == 0)
            return std::vector<Point<T>>();
        if (t == 1)
            return std::vector<Point<T>>{p};
        const ld k = sqrtl(r * r - d * d);
        return std::vector<Point<T>>{p - (l.v / l.v.len()) * k, p + (l.v / l.v.len()) * k};
    }
    // 圆与圆交点
    std::vector<Point<T>> inter(const Circle &a) const
    {
        const ld d = c.dis(a.c);
```

```
    const int t = relation(a);
    if (t == -1 || t == 0 || t == 4)
        return std::vector<Point<T>>();
    Point<T> e = a.c - c;
    e = e / e.len() * r;
    if (t == 1 || t == 3)
    {
        if (r * r + d * d - a.r * a.r >= -eps)
            return std::vector<Point<T>>{c + e};
        return std::vector<Point<T>>{c - e};
    }
    const ld costh = (r * r + d * d - a.r * a.r) / (2 * r * d),
                     sinth = sqrtl(1 - std::min(1.l, costh * costh));
    return std::vector<Point<T>>{c + e.rotate(costh, -sinth), c + e.rotate(costh, sinth)};
}
// 圆与圆交面积
ld inter_area(const Circle &a) const
{
    const ld d = c.dis(a.c);
    const int t = relation(a);
    if (t == -1)
        return area();
    if (t < 2)
        return 0;
    if (t > 2)
        return std::min(area(), a.area());
    const ld costh1 = (r * r + d * d - a.r * a.r) / (2 * r * d),
             costh2 = (a.r * a.r + d * d - r * r) / (2 * a.r * d);
    const ld sinth1 = sqrtl(1 - std::min(costh1 * costh1, 1.l)),
             sinth2 = sqrtl(1 - std::min(costh2 * costh2, 1.l));
    const ld th1 = acosl(costh1), th2 = acosl(costh2);
    return r * r * (th1 - costh1 * sinth1) + a.r * a.r * (th2 - costh2 * sinth2);
}
// 过圆外一点圆的切线
std::vector<Line<T>> tangent(const Point<T> &a) const
{
    const int t = is_in(a);
    if (t == 1)
        return std::vector<Line<T>>();
    if (t == -1)
    {
        const Point<T> v = {-(a - c).y, (a - c).x};
        return std::vector<Line<T>>{{a, v}};
    }
    Point<T> e = a - c;
    e = e / e.len() * r;
    const long double costh = r / c.dis(a), sinth = sqrtl(1 - std::min(costh * costh, 1.l));
    const Point<T> t1 = c + e.rotate(costh, -sinth), t2 = c + e.rotate(costh, sinth);
    return std::vector<Line<T>>{{a, t1 - a}, {a, t2 - a}};
}
// 两圆的公切线
std::vector<Line<T>> tangent(const Circle &a) const
{
    const int t = relation(a);
    std::vector<Line<T>> lines;
    if (t == -1 || t == 4)
        return lines;
    if (t == 1 || t == 3)
    {
        const Point<T> p = inter(a)[0], v = {-(a.c - c).y, (a.c - c).x};
        lines.push_back({p, v});
    }
    const ld d = c.dis(a.c);
    const Point<T> e = (a.c - c) / (a.c - c).len();
    if (t <= 2)
    {
        const ld costh = (r - a.r) / d, sinth = sqrtl(1 - std::min(costh * costh, 1.l));
        const Point<T> d1 = e.rotate(costh, -sinth), d2 = e.rotate(costh, sinth);
        const Point<T> u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c + d1 * a.r, v2 = a.c + d2 * a.r;
        lines.push_back({u1, v1 - u1});
        lines.push_back({u2, v2 - u2});
    }
    if (t == 0)
    {
        const ld costh = (r + a.r) / d, sinth = sqrtl(1 - std::min(costh * costh, 1.l));
        const Point<T> d1 = e.rotate(costh, -sinth), d2 = e.rotate(costh, sinth);
        const Point<T> u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c - d1 * a.r, v2 = a.c - d2 * a.r;
        lines.push_back({u1, v1 - u1});
        lines.push_back({u2, v2 - u2});
    }
```

```
        return lines;
    }
    // 直线的圆反演
    std::tuple<int, Circle, Line<T>> inverse(const Line<T> &l) const
    {
        const Circle null_c = {{0.0, 0.0}, 0.0};
        const Line<T> null_l = {{0.0, 0.0}, {0.0, 0.0}};
        if (l.toleft(c) == 0)
            return {2, null_c, l};
        const Point<T> v = l.toleft(c) == 1 ? Point<T>{l.v.y, -l.v.x} : Point<T>{-l.v.y, l.v.x};
        const ld d = r * r / l.dis(c);
        const Point<T> p = c + v / v.len() * d;
        return {1, {(c + p) / 2, d / 2}, null_l};
    }
    // 圆的圆反演
    std::tuple<int, Circle, Line<T>> inverse(const Circle &a) const
    {
        const Circle null_c = {{0.0, 0.0}, 0.0};
        const Line<T> null_l = {{0.0, 0.0}, {0.0, 0.0}};
        const Point<T> v = a.c - c;
        if (a.is_in(c) == -1)
        {
            const ld d = r * r / (a.r + a.r);
            const Point<T> p = c + v / v.len() * d;
            return {2, null_c, {p, {-v.y, v.x}}};
        }
        if (c == a.c)
            return {1, {c, r * r / a.r}, null_l};
        const ld d1 = r * r / (c.dis(a.c) - a.r), d2 = r * r / (c.dis(a.c) + a.r);
        const Point<T> p = c + v / v.len() * d1, q = c + v / v.len() * d2;
        return {1, {(p + q) / 2, p.dis(q) / 2}, null_l};
    }
};

// 圆与多边形面积交
template <class T>
ld area(const Circle<T> &circ, const Polygon<T> &poly)
{
    const auto cal = [](const Circle<T> &circ, const Point<T> &a, const Point<T> &b)
    {
        if ((a - circ.c).toleft(b - circ.c) == 0)
            return 0.0l;
        const auto ina = circ.is_in(a), inb = circ.is_in(b);
        const Line<T> ab = {a, b - a};
        if (ina && inb)
            return ((a - circ.c) % (b - circ.c)) / 2;
        if (ina && !inb)
        {
            const auto t = circ.inter(ab);
            const Point<T> p = t.size() == 1 ? t[0] : t[1];
            const ld ans = ((a - circ.c) % (p - circ.c)) / 2;
            const ld th = (p - circ.c).ang(b - circ.c);
            const ld d = circ.r * circ.r * th / 2;
            if ((a - circ.c).toleft(b - circ.c) == 1)
                return ans + d;
            return ans - d;
        }
        if (!ina && inb)
        {
            const Point p = circ.inter(ab)[0];
            const ld ans = ((p - circ.c) % (b - circ.c)) / 2;
            const ld th = (a - circ.c).ang(p - circ.c);
            const ld d = circ.r * circ.r * th / 2;
            if ((a - circ.c).toleft(b - circ.c) == 1)
                return ans + d;
            return ans - d;
        }
        const auto p = circ.inter(ab);
        if (p.size() == 2 && Segment<T>{a, b}.dis(circ.c) <= circ.r + eps)
        {
            const ld ans = ((p[0] - circ.c) % (p[1] - circ.c)) / 2;
            const ld th1 = (a - circ.c).ang(p[0] - circ.c), th2 = (b - circ.c).ang(p[1] - circ.c);
            const ld d1 = circ.r * circ.r * th1 / 2, d2 = circ.r * circ.r * th2 / 2;
            if ((a - circ.c).toleft(b - circ.c) == 1)
                return ans + d1 + d2;
            return ans - d1 - d2;
        }
        const ld th = (a - circ.c).ang(b - circ.c);
        if ((a - circ.c).toleft(b - circ.c) == 1)
            return circ.r * circ.r * th / 2;
```

```
        return -circ.r * circ.r * th / 2;
    };

    ld ans = 0;
    for (int i = 0; i < poly.p.size(); i++)
    {
        const Point<T> a = poly.p[i], b = poly.p[poly.nxt(i)];
        ans += cal(circ, a, b);
    }
    return ans;
}

// 多边形面积并
// 轮廓积分，复杂度 O(n^2logn)，n为边数
// ans[i] 表示被至少覆盖了 i+1 次的区域的面积
template <class T>
std::vector<ld> area(const std::vector<Polygon<T>> &polys)
{
    const int siz = polys.size();
    std::vector<std::vector<std::pair<Point<T>, Point<T>>>> segs(siz);
    const auto check = [](const Point<T> &u, const Segment<T> &e)
    { return !((u < e.a && u < e.b) || (u > e.a && u > e.b)); };

    auto cut_edge = [&](const Segment<T> &e, const int i)
    {
        const Line<T> le{e.a, e.b - e.a};
        std::vector<std::pair<Point<T>, int>> evt;
        evt.push_back({e.a, 0});
        evt.push_back({e.b, 0});
        for (int j = 0; j < polys.size(); j++)
        {
            if (i == j)
                continue;
            const auto &pj = polys[j];
            for (int k = 0; k < pj.p.size(); k++)
            {
                const Segment<T> s = {pj.p[k], pj.p[pj.nxt(k)]};
                if (le.toleft(s.a) == 0 && le.toleft(s.b) == 0)
                {
                    evt.push_back({s.a, 0});
                    evt.push_back({s.b, 0});
                }
                else if (s.isCross1(le))
                {
                    const Line<T> ls{s.a, s.b - s.a};
                    const Point<T> u = le.inter(ls);
                    if (le.toleft(s.a) < 0 && le.toleft(s.b) >= 0)
                        evt.push_back({u, -1});
                    else if (le.toleft(s.a) >= 0 && le.toleft(s.b) < 0)
                        evt.push_back({u, 1});
                }
            }
        }
        std::sort(evt.begin(), evt.end());
        if (e.a > e.b)
            std::reverse(evt.begin(), evt.end());
        int sum = 0;
        for (int i = 0; i < evt.size(); i++)
        {
            sum += evt[i].second;
            const Point<T> u = evt[i].first, v = evt[i + 1].first;
            if (!(u == v) && check(u, e) && check(v, e))
                segs[sum].push_back({u, v});
            if (v == e.b)
                break;
        }
    };

    for (int i = 0; i < polys.size(); i++)
    {
        const auto &pi = polys[i];
        for (int k = 0; k < pi.p.size(); k++)
        {
            const Segment<T> ei = {pi.p[k], pi.p[pi.nxt(k)]};
            cut_edge(ei, i);
        }
    }
    std::vector<ld> ans(siz);
    for (int i = 0; i < siz; i++)
    {
```

```
        ld sum = 0;
        std::sort(segs[i].begin(), segs[i].end());
        for (int j = 0; j < segs[i].size();)
        {
            int k = j;
            while (k < segs[i].size() && segs[i][k] == segs[i][j])
            {
                k++;
            }
            int count = k - j;
            sum += segs[i][j].first % segs[i][j].second;
            // 将多余的 count-1 个线段传递到下一层
            if (i + 1 < siz && count > 1)
            {
                for (int l = 0; l < count - 1; ++l)
                {
                    segs[i + 1].push_back(segs[i][j]);
                }
            }
            j = k;
        }
        ans[i] = sum / 2;
    }
    return ans;
}

// 圆面积并
// 轮廓积分，复杂度 O(n^2logn)
// ans[i] 表示被至少覆盖了 i+1 次的区域的面积
template <class T>
std::vector<ld> area(const std::vector<Circle<T>> &circs)
{
    const int siz = circs.size();
    using arc_t = std::tuple<Point<T>, ld, ld, ld>;
    std::vector<std::vector<arc_t>> arcs(siz);
    const auto eq = [](const arc_t &u, const arc_t &v)
    {
        const auto [u1, u2, u3, u4] = u;
        const auto [v1, v2, v3, v4] = v;
        return u1 == v1 && fabs(u2 - v2) <= eps && fabs(u3 - v3) <= eps && fabs(u4 - v4) <= eps;
    };

    auto cut_circ = [&](const Circle<T> &ci, const int i)
    {
        std::vector<std::pair<ld, int>> evt;
        evt.push_back({-pi, 0});
        evt.push_back({pi, 0});
        int init = 0;
        for (int j = 0; j < circs.size(); j++)
        {
            if (i == j)
                continue;
            const Circle<T> &cj = circs[j];
            if (ci.r < cj.r - eps && ci.relation(cj) >= 3)
                init++;
            const auto inters = ci.inter(cj);
            if (inters.size() == 1)
                evt.push_back({atan2l((inters[0] - ci.c).y, (inters[0] - ci.c).x), 0});
            if (inters.size() == 2)
            {
                const Point<T> dl = inters[0] - ci.c, dr = inters[1] - ci.c;
                long double argl = atan2l(dl.y, dl.x), argr = atan2l(dr.y, dr.x);
                if (std::fabs(argl + pi) <= eps)
                    argl = pi;
                if (std::fabs(argr + pi) <= eps)
                    argr = pi;
                if (argl > argr + eps)
                {
                    evt.push_back({argl, 1});
                    evt.push_back({pi, -1});
                    evt.push_back({-pi, 1});
                    evt.push_back({argr, -1});
                }
                else
                {
                    evt.push_back({argl, 1});
                    evt.push_back({argr, -1});
                }
            }
        }
```

```
        std::sort(evt.begin(), evt.end());
        int sum = init;
        for (int i = 0; i < evt.size(); i++)
        {
            sum += evt[i].second;
            if (std::fabs(evt[i].first - evt[i + 1].first) > eps)
                arcs[sum].push_back({ci.c, ci.r, evt[i].first, evt[i + 1].first});
            if (std::fabs(evt[i + 1].first - pi) <= eps)
                break;
        }
    };

    const auto oint = [](const arc_t &arc)
    {
        const auto [cc, cr, l, r] = arc;
        if (fabs(r - l - pi - pi) <= eps)
            return 2.0l * pi * cr * cr;
        return cr * cr * (r - l) + cc.x * cr * (sinl(r) - sinl(l)) - cc.y * cr * (cosl(r) - cosl(l));
    };

    for (int i = 0; i < circs.size(); i++)
    {
        const auto &ci = circs[i];
        cut_circ(ci, i);
    }
    std::vector<ld> ans(siz);
    for (int i = 0; i < siz; i++)
    {
        ld sum = 0;
        std::sort(arcs[i].begin(), arcs[i].end());
        for (int j = 0; j < arcs[i].size();)
        {
            int k = j;
            while (k < arcs[i].size() && eq(arcs[i][k], arcs[i][j]))
            {
                k++;
            }
            int count = k - j;
            sum += oint(arcs[i][j]);

            // 将多余的 count-1 个圆弧传递到下一层
            if (i + 1 < siz && count > 1)
            {
                for (int l = 0; l < count - 1; ++l)
                {
                    arcs[i + 1].push_back(arcs[i][j]);
                }
            }
            j = k;
        }
        ans[i] = sum / 2;
    }
    return ans;
}

// 半平面交
// 排序增量法，复杂度 O(nlogn)
// 输入与返回值都是用直线表示的半平面集合

// 有向线段的左边作为合法的半平面
template <class T>
std::vector<Line<T>> halfinter(std::vector<Line<T>> l, const ld lim = 1e18)
{
    const auto check = [](const Line<T> &a, const Line<T> &b, const Line<T> &c)
    { return a.toleft(b.inter(c)) < 0; };
    // // 无精度误差的方法，但注意取值范围会扩大到三次方
    // const auto check = [](const Line<T> &a, const Line<T> &b, const Line<T> &c)
    // {
    //     const Point<T> p = a.v * (b.v % c.v),
    //      q = b.p * (b.v % c.v) + b.v * (c.v % (b.p - c.p)) - a.p * (b.v % c.v);
    //     return p.toleft(q) < 0;
    // };
    l.push_back({{-lim, 0}, {0, -1}});
    l.push_back({{0, -lim}, {1, 0}});
    l.push_back({{lim, 0}, {0, 1}});
    l.push_back({{0, lim}, {-1, 0}});
    std::sort(l.begin(), l.end());
    std::deque<Line<T>> q;
    for (int i = 0; i < l.size(); i++)
    {
```

```
        if (i > 0 && l[i - 1].v.toleft(l[i].v) == 0 && l[i - 1].v * l[i].v > eps)
            continue;
        while (q.size() > 1 && check(l[i], q.back(), q[q.size() - 2]))
            q.pop_back();
        while (q.size() > 1 && check(l[i], q[0], q[1]))
            q.pop_front();
        if (!q.empty() && q.back().v.toleft(l[i].v) <= 0)
            return std::vector<Line<T>>();
        q.push_back(l[i]);
    }
    while (q.size() > 1 && check(q[0], q.back(), q[q.size() - 2]))
        q.pop_back();
    while (q.size() > 1 && check(q.back(), q[0], q[1]))
        q.pop_front();
    return std::vector<Line<T>>(q.begin(), q.end());
}

// 点集形成的最小最大三角形
// 极角序扫描线，复杂度 O(n%2logn)
// 最大三角形问题可以使用凸包与旋转卡壳做到 O(n%2)
template <class T>
std::pair<ld, ld> minmax_triangle(const std::vector<Point<T>> &vec)
{
    if (vec.size() <= 2)
        return {0, 0};
    std::vector<std::pair<int, int>> evt;
    evt.reserve(vec.size() * vec.size());
    ld maxans = 0, minans = std::numeric_limits<ld>::max();
    for (int i = 0; i < vec.size(); i++)
    {
        for (int j = 0; j < vec.size(); j++)
        {
            if (i == j)
                continue;
            if (vec[i] == vec[j])
                minans = 0;
            else
                evt.push_back({i, j});
        }
    }
    std::sort(evt.begin(), evt.end(), [&](const std::pair<int, int> &u, const std::pair<int, int> &v)
              {
        const Point du=vec[u.second]-vec[u.first],dv=vec[v.second]-vec[v.first];
        return argcmp<T>()({du.y,-du.x},{dv.y,-dv.x}); });
    std::vector<int> vx(vec.size()), pos(vec.size());
    for (int i = 0; i < vec.size(); i++)
        vx[i] = i;
    std::sort(vx.begin(), vx.end(), [&](int x, int y)
              { return vec[x] < vec[y]; });
    for (int i = 0; i < vx.size(); i++)
        pos[vx[i]] = i;
    for (auto [u, v] : evt)
    {
        const int i = pos[u], j = pos[v];
        const int l = std::min(i, j), r = std::max(i, j);
        const Point vecu = vec[u], vecv = vec[v];
        if (l > 0)
            minans = std::min(minans, fabs((vec[vx[l - 1]] - vecu) % (vec[vx[l - 1]] - vecv)));
        if (r < vx.size() - 1)
            minans = std::min(minans, fabs((vec[vx[r + 1]] - vecu) % (vec[vx[r + 1]] - vecv)));
        maxans = std::max({maxans, fabs((vec[vx[0]] - vecu) % (vec[vx[0]] - vecv)),
                           fabs((vec[vx.back()] - vecu) % (vec[vx.back()] - vecv))});
        if (i < j)
            std::swap(vx[i], vx[j]), pos[u] = j, pos[v] = i;
    }
    return {minans, maxans};
}

// 判断多条线段是否两两之间没有交点?
// 只要有任意一个交点 返回true 否则返回false(即两两都没有交)
// 扫描线，复杂度 O(nlogn)

// 可能存在精度问题 下面有更稳定的迭代器版本
template <class T>
bool inter(const std::vector<Segment<T>> &segs)
{
    if (segs.empty())
        return false;
    using seq_t = std::tuple<ld, int, Segment<T>>;
    const auto seqcmp = [](const seq_t &u, const seq_t &v)
```

```
    {
        const auto [u0, u1, u2] = u;
        const auto [v0, v1, v2] = v;
        if (fabs(u0 - v0) <= eps)
            return std::make_pair(u1, u2) < std::make_pair(v1, v2);
        return u0 < v0 - eps;
    };
    std::vector<seq_t> seq;
    for (auto seg : segs)
    {
        if (seg.a.x > seg.b.x + eps)
            std::swap(seg.a, seg.b);
        seq.push_back({seg.a.x, 0, seg});
        seq.push_back({seg.b.x, 1, seg});
    }
    std::sort(seq.begin(), seq.end(), seqcmp);
    ld x_now;
    auto cmp = [&](const Segment<T> &u, const Segment<T> &v)
    {
        if (fabs(u.a.x - u.b.x) <= eps || fabs(v.a.x - v.b.x) <= eps)
            return u.a.y < v.a.y - eps;
        return ((x_now - u.a.x) * (u.b.y - u.a.y) + u.a.y * (u.b.x - u.a.x)) * (v.b.x - v.a.x)
        < ((x_now - v.a.x) * (v.b.y - v.a.y) + v.a.y * (v.b.x - v.a.x)) * (u.b.x - u.a.x) - eps;
    };
    std::multiset<Segment<T>, decltype(cmp)> s{cmp};
    for (const auto [x, o, seg] : seq)
    {
        x_now = x;
        const auto it = s.lower_bound(seg);
        if (o == 0)
        {
            if (it != s.end() && seg.isCross2(*it))
                return true;
            if (it != s.begin() && seg.isCross2(*std::prev(it)))
                return true;
            s.insert(seg);
        }
        else
        {
            if (std::next(it) != s.end() && it != s.begin()
                            && (*std::prev(it)).isCross2(*std::next(it)))
                return true;
            s.erase(it);
        }
    }
    return false;
}
// // 判断多条线段是否两两之间存在交点（Lambda Map 最终版）
// template <class T>
// bool inter(const std::vector<Segment<T>> &segs)
// {
//     if (segs.empty())
//         return false;
//     using seq_t = std::tuple<ld, int, Segment<T>>;

//     // 1. 事件点创建和排序（不变）
//     std::vector<seq_t> seq;
//     for (auto seg : segs)
//     {
//         if (seg.a.x > seg.b.x + eps)
//             std::swap(seg.a, seg.b);
//         seq.push_back({seg.a.x, 0, seg});
//         seq.push_back({seg.b.x, 1, seg});
//     }
//     std::sort(seq.begin(), seq.end(), [](const seq_t &u, const seq_t &v)
//               {
//         if (fabs(std::get<0>(u) - std::get<0>(v)) <= eps)
//             return std::get<1>(u) < std::get<1>(v);
//         return std::get<0>(u) < std::get<0>(v) - eps; });

//     ld x_now;
//     // 2. multiset 的比较器（不变）
//     auto cmp = [&](const Segment<T> &u, const Segment<T> &v)
//     {
//         if (fabs(u.a.x - u.b.x) <= eps || fabs(v.a.x - v.b.x) <= eps)
//             return u.a.y < v.a.y - eps;
//         return ((x_now - u.a.x) * (u.b.y - u.a.y) + u.a.y * (u.b.x - u.a.x)) * (v.b.x - v.a.x)
//             < ((x_now - v.a.x) * (v.b.y - v.a.y) + v.a.y * (v.b.x - v.a.x)) * (u.b.x - u.a.x) - eps;
//     };
//     std::multiset<Segment<T>, decltype(cmp)> s{cmp};
```

```
//     auto C = [](const Segment<T> &l, const Segment<T> &r)
//     {
//         // 使用 std::tie 提供一个稳定的、唯一的排序规则
//         return std::tie(l.a.x, l.a.y, l.b.x, l.b.y) <
//                std::tie(r.a.x, r.a.y, r.b.x, r.b.y);
//     };

//     // 【改动 2】: 声明 map 时，将 lambda 的类型作为模板参数，
//     //             并将 lambda 对象本身作为构造函数参数
//     using m_iter = typename std::multiset<Segment<T>, decltype(cmp)>::iterator;
//     std::map<Segment<T>, m_iter, decltype(C)> seg_iters(C);

//     // ^^^^^^^^^^^^^^^^ 【核心修改在这里】 ^^^^^^^^^^^^^^^^

//     for (const auto &[x, o, seg] : seq)
//     {
//         x_now = x;
//         if (o == 0) // 处理左端点
//         {
//             auto it = s.insert(seg);
//             seg_iters[seg] = it; // map 现在可以正常工作

//             if (it != s.begin() && seg.isCross2(*std::prev(it)))
//                 return true;
//             if (std::next(it) != s.end() && seg.isCross2(*std::next(it)))
//                 return true;
//         }
//         else // 处理右端点
//         {
//             auto it = seg_iters.find(seg)->second; // 精确查找

//             auto prev_it = (it == s.begin()) ? s.end() : std::prev(it);
//             auto next_it = std::next(it);

//             if (prev_it != s.end() && next_it != s.end() && (*prev_it).isCross2(*next_it))
//             {
//                 return true;
//             }
//             s.erase(it); // 精确删除
//             seg_iters.erase(seg);
//         }
//     }
//     return false;
// }
// 推荐的 inCircle 实现 大整数实现
// 返回值:
// > 0: d在圆外
// < 0: d在圆内
// = 0: d在圆上
__int128 det(__int128 a, __int128 b, __int128 c,
             __int128 d, __int128 e, __int128 f,
             __int128 g, __int128 h, __int128 i)
{
    return a * (e * i - f * h) - b * (d * i - f * g) + c * (d * h - e * g);
}
template <class T>
int inCircle(Point<T> a, Point<T> b, Point<T> c, Point<T> d)
{
    // 平移 d 到原点，减少数值大小
    a = a - d;
    b = b - d;
    c = c - d;
    auto val = det(
        a.x, a.y, dis2(a),
        b.x, b.y, dis2(b),
        c.x, c.y, dis2(c));

    // 计算 a,b,c 的方向
    ll orient = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    if (orient < 0)
        val = -val; // 若顺时针则反转符号

    return (val > 0) - (val < 0); // 统一返回 -1/0/1
}
```

### 2.0.2 自适应辛普森法

自适应辛普森法的用途主要是高精度数值积分，即计算积分值。

```
using ld = long double;

// ====================== 非递归自适应辛普森法 ======================
// F: 被积函数类型
// a, b: 积分区间
// eps: 精度要求（可选）
template <typename F>
ld adaptiveSimpsonIter(F f, ld a, ld b, ld eps = 1e-12)
{
    struct Node
    {
        ld l, r; // 区间 [l, r]
        ld S;    // 该区间辛普森积分
        ld eps;  // 当前误差要求
    };

    auto simpson = [&](ld l, ld r) -> ld
    {
        ld m = (l + r) / 2;
        return (r - l) / 6 * (f(l) + 4 * f(m) + f(r));
    };

    std::stack<Node> stk;
    ld result = 0;

    // 初始化区间
    stk.push({a, b, simpson(a, b), eps});

    while (!stk.empty())
    {
        Node cur = stk.top();
        stk.pop();

        ld l = cur.l, r = cur.r, S = cur.S, e = cur.eps;
        ld m = (l + r) / 2;
        ld Sleft = simpson(l, m);
        ld Sright = simpson(m, r);

        // 精度满足，累加积分
        if (std::abs(Sleft + Sright - S) <= 15 * e)
        {
            result += Sleft + Sright + (Sleft + Sright - S) / 15.0;
        }
        else
        {
            // 左右子区间继续细分，误差按比例分配
            stk.push({m, r, Sright, e / 2});
            stk.push({l, m, Sleft, e / 2});
        }
    }

    return result;
}

// ====================== 示例使用 ======================
int main()
{
    auto f = [](ld x)
    {
        return std::sin(x);
    };
    ld res = adaptiveSimpsonIter(f, 0, acosl(-1));
    std::cout << std::setprecision(20) << res << "\n"; // 理论值为 2
}
```

### 2.0.3 平面最近点对

```
static inline ll sq(ll v) { return v * v; }
// 计算两点平方距离
static inline ll dist2(const Point &a, const Point &b)
{
    return sq(a.x - b.x) + sq(a.y - b.y);
}
void solve(std::vector<std::pair<int, int>> &p)
```

```
{
    int n = p.size();
    // idx: 以 x 优先、y 次之排序后的点编号（用于按 x 分治的“切分边界”）
    std::vector<int> idx(n);
    std::iota(idx.begin(), idx.end(), 0);
    std::sort(idx.begin(), idx.end(), [&](int i, int j)
              {
        if (p[i].x != p[j].x) return p[i].x < p[j].x;
        return p[i].y < p[j].y; });

    // ord: 与 idx 同一集合，但每段在递归返回时保持按 y 升序
    std::vector<int> ord = idx;
    // 分治：在 [l, r) 这段（元素集合等于 idx[l..r)），返回最小平方距离；
    // 递归结束时，保证 ord[l..r) 按 y 升序，便于父层用 inplace_merge 合并
    std::function<ll(int, int)> dc = [&](int l, int r) -> ll
    {
        int mlen = r - l;
        if (mlen <= 25)
        { // 小块直接暴力 + 就地按 y 排序
            std::sort(ord.begin() + l, ord.begin() + r, [&](int i, int j)
                      {
                if (p[i].y != p[j].y) return p[i].y < p[j].y;
                return p[i].x < p[j].x; });
            ll best = 9e18;
            for (int i = l; i < r; ++i)
                for (int j = i + 1; j < r; ++j)
                {
                    // 利用 y 排序做个小剪枝
                    ll dy = p[ord[j]].y - p[ord[i]].y;
                    if (dy * dy >= best)
                        break;
                    best = std::min(best, dist2(p[ord[i]], p[ord[j]]));
                }
            return best;
        }
        int m = (l + r) >> 1;
        // 中线 x（按 idx 的中点划分）
        ll midx = p[idx[m]].x;

        ll dl = dc(l, m);
        ll dr = dc(m, r);
        ll best = std::min(dl, dr);

        // 合并子段的“按 y 升序”到父段：用内置 inplace_merge
        std::inplace_merge(ord.begin() + l, ord.begin() + m, ord.begin() + r, [&](int i, int j)
                           {
            if (p[i].y != p[j].y) return p[i].y < p[j].y;
            return p[i].x < p[j].x; });

        // 构建条带：ord[l..r) 已按 y 升序
        std::vector<int> strip;
        strip.reserve(r - l);
        for (int k = l; k < r; ++k)
        {
            int id = ord[k];
            if (sq(p[id].x - midx) < best)
            {
                // 只需与前面最多 7~8 个候选比
                for (int t = (int)strip.size() - 1, cnt = 0; t >= 0 && cnt < 8; --t, ++cnt)
                {
                    best = std::min(best, dist2(p[id], p[strip[t]]));
                }
                strip.push_back(id);
            }
        }
        return best;
    };
    return dc(0, n);
}
```

### 2.0.4 最小圆覆盖和最小球覆盖

```
using ll = long long;
using ld = long double;

const ld PI = acosl(-1);
const ld EPS = 1e-12;
```

```cpp
struct Point
{
    ld x, y;
    // 向量加减
    Point operator+(const Point &o) const { return {x + o.x, y + o.y}; }
    Point operator-(const Point &o) const { return {x - o.x, y - o.y}; }
    Point operator*(ld k) const { return {x * k, y * k}; }
    Point operator/(ld k) const { return {x / k, y / k}; }
};
// 向量叉积
inline ld cross(const Point &a, const Point &b) { return a.x * b.y - a.y * b.x; }
// 向量旋转
inline Point rotate(const Point &a, ld ang)
{
    return {a.x * cosl(ang) - a.y * sinl(ang), a.x * sinl(ang) + a.y * cosl(ang)};
}
// 距离
inline ld dist(const Point &a, const Point &b)
{
    return std::hypotl(a.x - b.x, a.y - b.y);
}
struct Circle
{
    Point c; // 圆心
    ld r;    // 半径
};
// 直线交点：a + t*u 与 b + s*v
Point line_intersection(Point a, Point u, Point b, Point v)
{
    ld t = cross(b - a, v) / cross(u, v);
    return a + u * t;
}
std::pair<Point, Point> midperp(const Point &a, const Point &b)
{
    return {(a + b) / 2.0L, rotate(b - a, PI / 2)};
}
// 两点圆
Circle circle_from(const Point &a, const Point &b)
{
    return {(a + b) / 2.0L, dist(a, b) / 2.0L};
}
// 三点外接圆
Circle circle_from(const Point &a, const Point &b, const Point &c)
{
    auto u = midperp(a, b);
    auto v = midperp(a, c);
    Point o = line_intersection(u.first, u.second, v.first, v.second);
    return {o, dist(o, a)};
}
// 随机增量法
Circle work(std::vector<Point> &p)
{
    std::mt19937_64 rng((unsigned long long)(new char));
    std::shuffle(p.begin(), p.end(), rng);
    Circle C{p[0], 0};
    int n = (int)p.size();
    for (int i = 1; i < n; ++i)
    {
        if (dist(C.c, p[i]) <= C.r + EPS)
            continue;
        C = {p[i], 0};
        for (int j = 0; j < i; ++j)
        {
            if (dist(C.c, p[j]) <= C.r + EPS)
                continue;
            C = circle_from(p[i], p[j]);
            for (int k = 0; k < j; ++k)
            {
                if (dist(C.c, p[k]) <= C.r + EPS)
                    continue;
                C = circle_from(p[i], p[j], p[k]);
            }
        }
    }
    return C;
}
```

```cpp
using ll = long long;
using ld = long double;
```

```
const ld EPS = 1e-12;

struct P
{
    ld x, y, z;
    P operator+(const P &o) const { return {x + o.x, y + o.y, z + o.z}; }
    P operator-(const P &o) const { return {x - o.x, y - o.y, z - o.z}; }
    P operator*(ld k) const { return {x * k, y * k, z * k}; }
    P operator/(ld k) const { return {x / k, y / k, z / k}; }
};
inline ld dot(const P &a, const P &b)
{
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
inline P cross(const P &a, const P &b)
{
    return {a.y * b.z - a.z * b.y,
            a.z * b.x - a.x * b.z,
            a.x * b.y - a.y * b.x};
}
inline ld norm2(const P &a) { return dot(a, a); }
inline ld dist(const P &a, const P &b) { return std::sqrt(norm2(a - b)); }
struct Sphere
{
    P c;
    ld r;
};
inline bool check(const Sphere &s, const P &p)
{
    return dist(s.c, p) <= s.r + EPS;
}
// 两点球
Sphere calc(const P &a, const P &b)
{
    P o = (a + b) / 2.0L;
    return {o, dist(a, b) / 2.0L};
}

// 三点球（唯一确定一个外接圆，球心在平面内）
Sphere calc(const P &a, const P &b, const P &c)
{
    P ab = b - a, ac = c - a;
    P abXac = cross(ab, ac);
    ld d = 2 * norm2(abXac);
    if (d < EPS)
    { // 三点共线
        Sphere s1 = calc(a, b);
        Sphere s2 = calc(a, c);
        Sphere s3 = calc(b, c);
        Sphere res = s1;
        if (s2.r > res.r)
            res = s2;
        if (s3.r > res.r)
            res = s3;
        return res;
    }
    P ab_mid = (a + b) / 2.0L;
    P ac_mid = (a + c) / 2.0L;
    P n = abXac;

    // 球心计算公式
    P num = cross(abXac, ab) * norm2(ac) - cross(abXac, ac) * norm2(ab);
    P o = a + num / d;
    return {o, dist(o, a)};
}

// 四点外接球
Sphere calc(const P &a, const P &b, const P &c, const P &d)
{
    ld a1 = b.x - a.x, a2 = b.y - a.y, a3 = b.z - a.z;
    ld b1 = c.x - a.x, b2 = c.y - a.y, b3 = c.z - a.z;
    ld c1 = d.x - a.x, c2 = d.y - a.y, c3 = d.z - a.z;

    ld a_len = norm2(b) - norm2(a);
    ld b_len = norm2(c) - norm2(a);
    ld c_len = norm2(d) - norm2(a);

    ld det = a1 * (b2 * c3 - b3 * c2) - a2 * (b1 * c3 - b3 * c1)
+ a3 * (b1 * c2 - b2 * c1);
```

```
    if (fabsl(det) < EPS)
    {
        // 退化成三点球
        Sphere s1 = calc(a, b, c);
        if (!check(s1, d))
        {
            Sphere s2 = calc(a, b, d);
            if (!check(s2, c))
            {
                Sphere s3 = calc(a, c, d);
                if (!check(s3, b))
                {
                    return calc(b, c, d);
                }
                return s3;
            }
            return s2;
        }
        return s1;
    }
    P o;
    o.x = (a_len * (b2 * c3 - b3 * c2) - b_len * (a2 * c3 - a3 * c2)
            + c_len * (a2 * b3 - a3 * b2)) / (2 * det);
    o.y = (a_len * (b3 * c1 - b1 * c3) - b_len * (a3 * c1 - a1 * c3)
            + c_len * (a3 * b1 - a1 * b3)) / (2 * det);
    o.z = (a_len * (b1 * c2 - b2 * c1) - b_len * (a1 * c2 - a2 * c1)
            + c_len * (a1 * b2 - a2 * b1)) / (2 * det);

    return {o, dist(o, a)};
}
// 随机增量法
Sphere work(std::vector<P> pts)
{
    std::mt19937_64 rng((unsigned long long)(new char));
    std::shuffle(pts.begin(), pts.end(), rng);
    Sphere s{pts[0], 0};
    int n = pts.size();
    for (int i = 1; i < n; ++i)
    {
        if (check(s, pts[i]))
            continue;
        s = {pts[i], 0};
        for (int j = 0; j < i; ++j)
        {
            if (check(s, pts[j]))
                continue;
            s = calc(pts[i], pts[j]);
            for (int k = 0; k < j; ++k)
            {
                if (check(s, pts[k]))
                    continue;
                s = calc(pts[i], pts[j], pts[k]);
                for (int l = 0; l < k; ++l)
                {
                    if (check(s, pts[l]))
                        continue;
                    s = calc(pts[i], pts[j], pts[k], pts[l]);
                }
            }
        }
    }
    return s;
}
```